ELEN90098 REINFORCEMENT LEARNING FOR ENGINEERING
WORKSHOP 4: POLICY GRADIENT

SEMESTER 2, 2025

This workshop introduces you to the Policy Gradient algorithm. You will investigate the characteristics of these algorithms and evaluate their performance.

Submission: Each group must upload 2 files:

(1) A pdf (`.pdf`) file containing your answers to Task 1. Note that:
- Only one member of the group needs to upload this file;
- The naming convention of the file is `Workshop4-<LastName1>-<LastName2>.pdf` for a group whose members have last names, LastName1 and LastName2.

(2) A zip (`.zip`) file compressing your Policy Gradient code, with the `(TODO)` blanks all filled. Note that:
- Only one member of the group needs to upload this file;
- The naming convention of the file is `Workshop4-Code-<LastName1>-<LastName2>.zip` for a group whose members have last names, LastName1 and LastName2.

Note: Task 1 needs to be answered in this week's submission. The Final Report should include answers for both Task 1 and Task 2.

1. Introduction

Policy Gradient methods take a different approach to reinforcement learning by directly learning a parameterized policy $\pi_\theta(a|s)$. Instead of relying on tables or neural networks to estimate value functions like $Q(s, a)$, Policy Gradient optimizes the policy itself by adjusting its parameters θ in the direction that increases the expected return. This makes Policy Gradient a powerful solution in environments with continuous or high-dimensional action spaces, where value-based methods struggle.

In this assignment, you will implement and evaluate Policy Gradient for the continuous environment. The algorithm was introduced in the lecture, and starter code will be provided. Running on a GPU is recommended for efficiency.

We will test the algorithms on the `InvertedPendulum` environment. Up next, Section 2 provides an overview of the environment and a review of Policy Gradient. Section 3 details the tasks for this week.

2. Review

2.1. Environment: Inverted Pendulum.

This environment is the continuous version of CartPole, powered by the Mujoco physics simulator, allowing for more complex experiments. (Gymnasium website)

Action Space The agent takes a 1-element vector for actions. The action space is continuous in $[-3, 3]$, where the value represents the directional force applied to the cart.

Observation Space

The observation space is a `Box(-Inf, Inf, (4,), float64)` where the elements are as follows:

Rewards The goal is to keep the inverted pendulum stand upright ($|angle| < 0.2$) for as long as possible - as such, a reward of $+1$ is given for each timestep that the pole is upright.

Termination

The environment terminates when any of the following happens:

- Any of the state space values is no longer finite.

Idx	Observation	Min	Max
0	Cart Position (m)	$-\infty$	∞
1	Pole Angle (rad)	$-\infty$	∞
2	Cart Velocity (m/s)	$-\infty$	∞
3	Pole Angular Velocity (rad/s)	$-\infty$	∞

- Large Pole angle $|angle| \geq 0.2$
- Episode length is greater than 1000

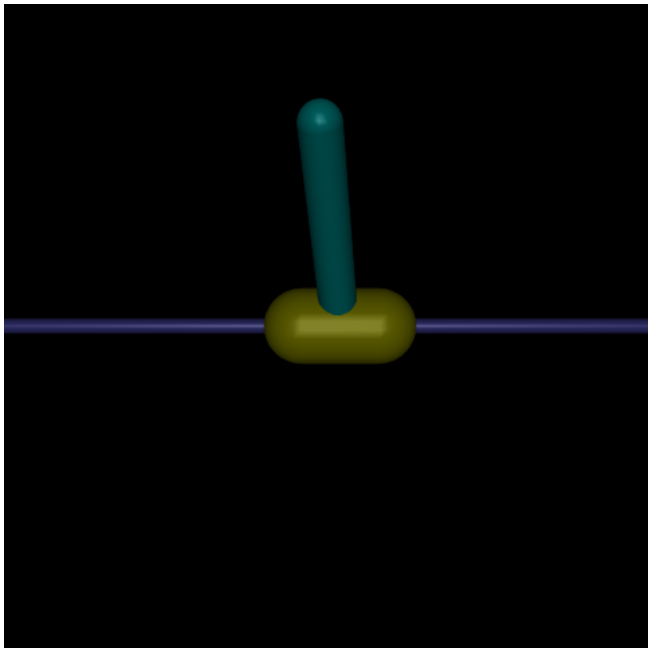


Figure 1. InvertedPendulum-v5

2.2. **Policy Gradient.** Policy gradient methods learn a parameterized policy $\pi_\theta(a|s)$ that maps states to a probability distribution over actions. Instead of estimating a value function to derive a policy, these methods update the policy parameters θ by directly maximizing the expected cumulative return through gradient ascent. The policy gradient theorem provides an unbiased estimator of the gradient of the expected return, allowing updates using trajectories collected from the current policy.

Unlike value-based algorithms such as Q-learning, policy gradient methods are typically on-policy, requiring data sampled from the very policy being optimized. This direct optimization is well-suited for environments with continuous or high-dimensional action spaces, where value-based approaches struggle to discretize or evaluate all possible actions.

A key challenge is the high variance of the gradient estimate. To address this, variance-reduction techniques are used:

- **Reward-to-Go:** Instead of using the total return R for every time step, we use the future return starting from time t, which respects causality and lowers variance.
- **Baseline:** A baseline function $b(s_t)$, often chosen as a state-value estimate, is subtracted from the return without introducing bias, further stabilizing learning.

These techniques strike a balance between bias and variance, enabling more efficient and stable policy updates.

3. Tasks

File overview and Implementation. The starter code is provided with blanks marked as `(TODO)` for you to complete in order to run the algorithm. Move the code under the workshop folder `rl-workshop` that you have created.

The files are organized as follows:

- `Policy_Gradient/`
  - `run_policy_gradient.py`
  - `policy_gradient/`
    - `agent.py`
    - `network.py`
  - `utils/`
    - `log_utils.py`
    - `logger.py`
    - `util.py`

Your work is to complete the `(TODO)` blanks in `run_policy_gradient.py` and `agent.py` to implement Policy Gradient. Before running, navigate to the workshop folder:

```
cd path/to/rl-workshop
```

The starter code automatically logs results to TensorBoard. To visualize training curves, run:

```
tensorboard --logdir ./logs
```

Then open `localhost:6006` in your browser.

—

Task 1: Inverted Pendulum (7 points). This task introduces you to Policy Gradient and variance reduction techniques: You will apply these algorithms to `InvertedPendulum` and analyze their learning behaviors.

Question 1 (2 points). What are the state space, action space, and reward structure of the `InvertedPendulum` environment? Are they finite or infinite? Discrete or continuous? Is the environment deterministic or stochastic?

Question 2 (5 points). For the `InvertedPendulum` environment, run the vanilla policy gradient algorithm and the variance reduction techniques (Reward-to-go, Baseline).

- Use TensorBoard to visualize training progress.
- Submit a plot showing evaluation returns (y-axis) versus environment steps (x-axis).
- Briefly comment on whether the learning curve indicates successful training.
- Is your policy deterministic or stochastic?

Commands:

Policy Gradient:

```
python Policy_Gradient/run_policy_gradient.py --env_name InvertedPendulum-v5 \
--log_video
```

Reward-to-go:

```
python Policy_Gradient/run_policy_gradient.py --env_name InvertedPendulum-v5 \
--log_video --use_reward_to_go --exp_name rtg
```

Baseline:

```
python Policy_Gradient/run_policy_gradient.py --env_name InvertedPendulum-v5 \
--log_video --use_baseline --exp_name baseline
```

Note: Policy gradient with variance reduction can solve `InvertedPendulum`, though it is relatively unstable and requires a well-tuned combination of hyperparameters.

—

Task 2. Policy Gradient

Question 1. Investigate how the learning rate α affects training outcomes. Train the agent with at least three different learning rates in the range $[10^{-5}, 10^{-2}] \subset \mathbb{R}$. Example command for $\alpha = 10^{-3}$, remember to change the `exp_name` according to your selected learning rate:

```
python Policy_Gradient/run_policy_gradient.py --env_name InvertedPendulum-v5 \
--log_video --learning_rate 0.001 --exp_name lr1e-3
```

Submit:

- A plot showing evaluation returns (y-axis) versus environment steps (x-axis) for each learning rate.

Discuss

- Which learning rate achieves the fastest convergence?
- How does the learning rate interact with variance reduction techniques (reward-to-go, baseline)? Does a higher learning rate amplify or mitigate their effect?
- Discuss whether the effect is related to the variance of the policy gradient estimates. For example, does a larger learning rate amplify the noise in high-variance updates, making convergence unstable, or can it still speed up learning when variance is reduced (e.g., by reward-to-go or baseline)? Explain your reasoning based on your plots and observed training behavior.

Question 2. Explore how the number of steps per batch influences training performance. The number of batch steps determines how many environment steps are collected before each policy update. Train the agent with at least three different values of `batch_steps`. Then repeat the experiments with each variance–reduction technique: reward-to-go, baseline.

Example command for `batch_steps=2000` (remember to change `exp_name` for each run):

```
python Policy_Gradient/run_policy_gradient.py --env_name InvertedPendulum-v5 \
--log_video --batch_steps 2000 --exp_name batch2k
```

Submit:

- A plot of evaluation returns (y-axis) versus environment steps (x-axis) with all batch steps, for each variance–reduction setting, totaling 3 figures.

Discuss

- For a fixed number of environment steps, does increasing the batch steps consistently improve the final evaluation return?
- More batch steps reduce the variance of each gradient estimate, but require more computation per update. How does this trade-off appear in your results (e.g., sample efficiency)?
- For each variance–reduction technique, does increasing batch steps reduce gradient variance as strongly as in vanilla Policy Gradient? Identify one technique where the benefit of more batch steps appears smallest.

Question 3. Run the Policy Gradient algorithm on `CartPole` and compare its learning performance with DQN. You will need to tune the hyperparameters (`total_steps, batch_steps, learning_rate, discount_gamma, network size`) to improve the performance.

- Submit a plot showing evaluation returns (y-axis) versus environment steps (x-axis) for both PG and DQN.
- Discuss: Which algorithm converges faster? What might explain the difference in learning speed?

Question 4. Apply the Policy Gradient algorithm to train a controller for the `HalfCheetah` environment. You will need to tune the hyperparameters to improve the performance.

- Use TensorBoard to visualize training progress.
- Submit a plot showing evaluation returns (y-axis) versus environment steps (x-axis).

Open the *Video* tab to watch the trained agent in action.

Command:

```
python Policy_Gradient/run_policy_gradient.py --env_name HalfCheetah-v5 \
--log_video --exp_name cheetah
```

—

Note: Task 1 must be answered this week. Task 2 must be answered in the final report. Answers to all tasks **must be included and discussed in your final report.**